

IoT Ergonomic Posture Monitor

Aaryans Nepal

CSC 494 - IoT (Spring 2026) — Sprint Week 6

Why the Pivot?

The original plan was a **theft detection system** that tracked hand-to-hip movements to flag concealment behavior. After weeks of research and prototyping, two fundamental problems killed it:

- **False positive nightmare:** Reaching into your own pocket, adjusting your belt, holding a phone on your hip — all trigger the same geometry as "concealment." No way to distinguish intent.
- **No face recognition:** Without identifying *who* is doing what, the system can't tell a shoplifter from an employee restocking shelves. Could have triggered snapshots, but the entire project felt like overkill for a dead-end idea.

The core tech (MediaPipe Pose on Pi 5) was solid — the *application* was flawed. So I kept the stack and pivoted to a different problem.

New Direction: Posture Monitoring

Problem: People working at desks slouch without realizing it, leading to chronic back and neck pain. I have this problem myself — I spend a lot of time at my desk in front of a PC/laptop, and my posture has gotten bad. But one thing I notice is that every time I *remember* my bad posture, I straighten up. So what if I built something that monitors my posture and reminds me every time I slouch?

Solution: A camera-based system on a **Raspberry Pi 5** that watches your posture in real-time, scores it, and buzzes you when you've been slouching too long.

- Runs entirely on-device
- Uses the same MediaPipe Pose model, just different math on top
- Hardware: Pi 5, Pi Camera Module 2, active buzzer on GPIO 17

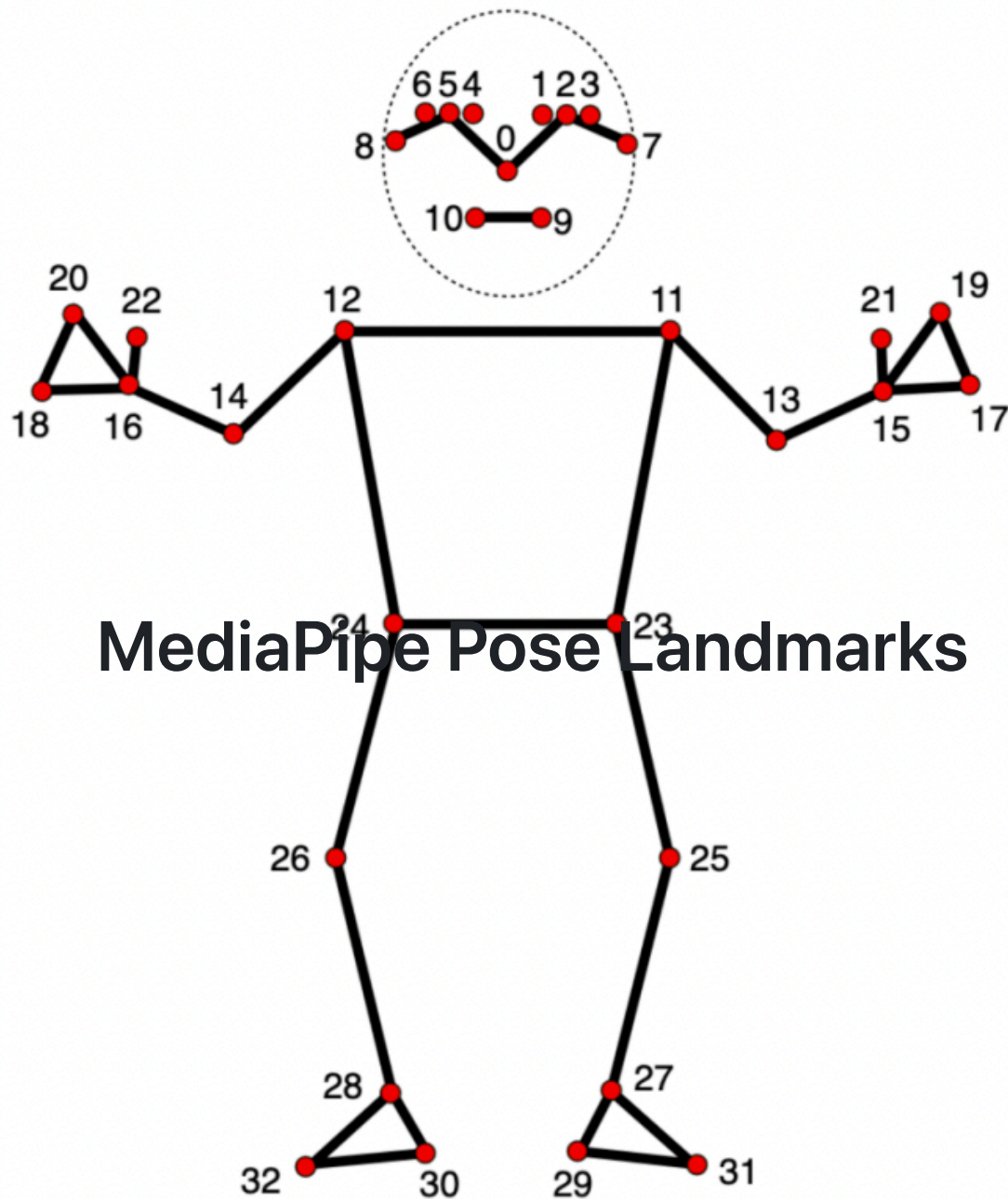
What I've Learned: MediaPipe Pose

MediaPipe's **BlazePose** model is a two-stage neural network:

1. **Detector**: Scans the full frame to find a human body (runs on first frame only)
2. **Landmark Network**: Predicts **33 body joints** from a cropped region around the person

On subsequent frames, it **skips the detector** and tracks from the previous position — this is why it runs at **8-12 FPS** on a Pi 5 CPU with no GPU.

Each landmark gives: **x, y** (normalized 0-1), **z** (depth relative to hips), and **visibility** (confidence 0-1).



- | | |
|-----------------------|-----------------------|
| 0 - nose | 17 - left pinky |
| 1 - left eye (inner) | 18 - right pinky |
| 2 - left eye | 19 - left index |
| 3 - left eye (outer) | 20 - right index |
| 4 - right eye (inner) | 21 - left thumb |
| 5 - right eye | 22 - right thumb |
| 6 - right eye (outer) | 23 - left hip |
| 7 - left ear | 24 - right hip |
| 8 - right ear | 25 - left knee |
| 9 - mouth (left) | 26 - right knee |
| 10 - mouth (right) | 27 - left ankle |
| 11 - left shoulder | 28 - right ankle |
| 12 - right shoulder | 29 - left heel |
| 13 - left elbow | 30 - right heel |
| 14 - right elbow | 31 - left foot index |
| 15 - left wrist | 32 - right foot index |
| 16 - right wrist | |

What I've Learned: The Metrics Problem

My first attempt used **X/Y coordinates** to measure slouching — head forward offset, head drop, shoulder tilt. The posture score was stuck at **0-2 out of 100** no matter how badly I slouched.

Why? With a front-facing camera, slouching is movement along the **Z-axis** (toward the camera). The head barely moves left/right or up/down in frame — it moves *closer*.

Built a **diagnostic script** that sampled my test video at 5 timepoints and printed every metric. The data showed:

- `head_forward` (X-based) changed by **0.003** — useless
- `nose_y` (absolute Y) changed by **0.103** — significant
- `nose_z_depth` changed by **0.062** — significant
- `face_size` **doubled** — the clearest signal of all

What I've Built: The Working System

5 data-driven metrics (replaced the original broken ones):

1. **nose_y**: Absolute Y position — drops when body sinks in frame
2. **nose_z_depth**: Nose Z minus shoulder Z — more negative = leaning forward
3. **ear_z_depth**: Ear Z minus shoulder Z — detects forward head posture
4. **face_size**: Distance from nose to ears — grows when closer to camera
5. **shoulder_tilt**: Left vs right shoulder Y — detects sideways leaning

Each metric is weighted and combined into a **posture score (0-100)**.

What I've Built: Calibration & Alerts

Personal Baseline Calibration: At startup, you sit up straight for 5 seconds. The system captures your personal "zero point" for all 5 metrics. All scoring is relative to *your* body — no universal thresholds.

State Machine (prevents false alarms from momentary movements):

- GOOD → sustained bad for **10 seconds** → ALERT (buzzer on)
- ALERT → sustained good for **3 seconds** → back to GOOD

Flask Web Stream: Live MJPEG stream at `http://<pi-ip>:5000` showing the camera feed with skeleton overlay, color-coded score bar, and posture status label.

What I've Built: Tech Stack

Component	Technology
Pose Estimation	MediaPipe Pose (BlazePose)
Video Processing	OpenCV
Web Streaming	Flask (MJPEG)
Physical Alert	Active buzzer via gpiozero (GPIO 17)
Event Logging	SQLite + JSON log
Remote Access	ZeroTier VPN
Platform	Raspberry Pi 5 (4GB RAM, 100GB SSD)

All processing is **local** — no cloud APIs, no data leaves the device.

Current Status

Working:

- MediaPipe running on Pi 5 at 8-12 FPS
- 5 validated posture metrics with personal calibration
- State machine with buzzer alerts
- Flask live stream with skeleton overlay and score display
- Diagnostic tooling for metric validation

Temporary workaround: Using phone-recorded video as input — Pi Camera adapter (15-pin to 22-pin) hasn't arrived yet.

Learning With AI

Claude has been great for coding assistance, but expensive.

Biggest lesson: All AI models are great "yes men." They agree with you, make you feel like a genius, and always seem to have the perfect paper or solution for your problem. But it took me one prompt — just saying "this feels like overkill with no real endpoint" — and the chatbot completely flipped, agreeing and listing every flaw the project had. It felt strange because it had been telling me the project was great the whole time.

Plan: Weeks 7-10

- **Camera Integration:** Switch from video file to live Pi Camera Module 2 feed once adapter arrives.
- **Data Collection Mode:** Record labeled posture sessions (good/bad) to build a training dataset.
- **ML Classifier:** Train a **scikit-learn** model on collected data to replace hand-tuned thresholds — more accurate, adapts to patterns the heuristics miss.
- **Dashboard Polish:** Add session history, posture trend graphs, and configurable sensitivity to the Flask UI.

Plan: Weeks 11-12

- **Benchmarking:** Measure FPS, accuracy (% correct classifications), and thermal performance over extended sessions.
- **Stretch Goals:** Add mouth/eye landmarks for head rotation and downward tilt detection; elbow tracking if visible.
- **Final Presentation:** Full demo with live camera, documented results, and lessons learned.

Key Takeaways So Far

- **Measure before you code:** My first metrics were based on assumptions. A 20-line diagnostic script showed they were all wrong.
- **Camera geometry matters:** Front-facing vs side-facing cameras capture completely different axes of movement — this changes which math works.
- **Edge AI is viable:** A \$60 Pi 5 runs production ML models at usable framerates with no GPU and no cloud.
- **Pivoting isn't failure:** The theft detection idea was flawed at a fundamental level. Recognizing that early saved weeks of wasted work.

